

Análise de Classificação Multiclasse: Perceptron vs. KNN

João Victor Simões Rosa

3 de maio de 2026

1 Introdução

A classificação multiclasse é um problema central da Aprendizagem de Máquina, no qual o objetivo é atribuir cada instância a uma entre três ou mais classes possíveis [1, 2]. Diferentes algoritmos resolvem essa tarefa a partir de hipóteses distintas sobre a organização dos dados, o que influencia diretamente o tipo de fronteira de decisão produzida e o desempenho obtido.

Neste relatório, analisamos dois métodos clássicos e conceitualmente contrastantes: o Perceptron [3], um classificador linear baseado no ajuste iterativo de pesos, e o k -Nearest Neighbors (KNN) [4], um método não paramétrico fundamentado em proximidade entre instâncias. A escolha desses algoritmos permite comparar uma abordagem global, que aprende explicitamente uma fronteira de decisão, com uma abordagem local, que decide com base na vizinhança dos dados.

Além da descrição teórica e análise crítica de cada método, avaliamos seu desempenho por meio da curva ROC e da métrica AUC, discutindo suas extensões para o contexto multiclasse [5, 6].

Este relatório está organizado da seguinte forma: a Subseção 2.1 apresenta o Perceptron, a Subseção 2.2 discute o KNN, a Subseção 2.3 compara os principais hiperparâmetros dos dois modelos e a Subseção 2.4 aborda as métricas de avaliação ROC e AUC. Por fim, a Seção de Resultados 3 apresenta os experimentos práticos e a Conclusão 4 sintetiza os principais aprendizados do estudo.

2 Desenvolvimento

2.1 Perceptron

De forma geral, o Perceptron é um modelo linear supervisionado que busca aprender uma função que mapeia as entradas para os rótulos de saída correspondentes. O funcionamento do algoritmo baseia-se no ajuste iterativo de um vetor de pesos: a cada época (*epoch*) do treinamento, o modelo realiza predições e, caso cometa erros, atualiza os pesos visando reduzir a taxa de classificações incorretas.

A eficácia do Perceptron depende criticamente da configuração de seus hiperparâmetros, que serão apresentados de forma comparativa na Subseção 2.3.

Geometricamente, a fronteira de decisão do Perceptron é estritamente linear (ou um hiperplano em dimensões superiores). Isso ocorre porque a regra de decisão é uma função linear dos atributos ($f(x) = w \cdot x + b$). Consequentemente, o modelo só é capaz de separar classes que sejam *linearmente separáveis*.

O hiperparâmetro `fit_intercept` é crucial aqui: quando *False*, a fronteira é forçada a passar pela origem $(0, 0)$, o que restringe severamente a capacidade do modelo. Quando *True*, a fronteira ganha liberdade de translação no espaço, mas mantém sua natureza linear.

Apesar de eficiente em cenários lineares, o Perceptron falha em convergir adequadamente quando a complexidade da distribuição dos dados exige fronteiras curvas ou mais elaboradas. Para contornar essa limitação, recorreremos a métodos não lineares, como o KNN, que será abordado a seguir.

2.2 k -Nearest Neighbors (KNN)

Diferentemente do Perceptron, o k -Nearest Neighbors (KNN) é um algoritmo de aprendizado baseado em instâncias e não paramétrico. Isso significa que ele não tenta aprender uma função matemática global durante uma etapa de treinamento; em vez disso, o KNN "memoriza" os dados de treinamento e adia o processamento para o momento da classificação. Quando um novo dado é inserido, o algoritmo

calcula a distância entre ele e todos os outros pontos do conjunto de treinamento para identificar os k vizinhos mais próximos [4, 2].

A decisão da classe final ocorre por meio de uma votação majoritária entre esses k vizinhos. Por depender de cálculos de proximidade, o desempenho do KNN é altamente sensível à escala dos dados e à métrica de distância escolhida. Seus principais hiperparâmetros também serão apresentados na Subseção 2.3.

2.2.1 Fronteira de Decisão e Flexibilidade

A fronteira de decisão do KNN é inerentemente **não linear**. Isso ocorre porque ela não é definida por uma equação fixa, mas pela distribuição espacial das amostras. Geometricamente, a fronteira é formada por contornos que se adaptam ao formato dos agrupamentos de dados, permitindo que o modelo capture relações complexas e "ilhas" de classes que um modelo linear jamais conseguiria isolar.

Enquanto o Perceptron falha em dados que não podem ser separados por uma reta, o KNN consegue se moldar a essas distribuições. No entanto, essa flexibilidade tem um custo: o KNN é computacionalmente caro para grandes volumes de dados, pois precisa calcular distâncias para cada nova predição, e sua eficácia diminui em espaços com muitas dimensões (*curse of dimensionality*).

2.3 Comparação de Hiperparâmetros

A Tabela 1 resume os principais hiperparâmetros do Perceptron e do KNN, bem como seus impactos no comportamento de cada modelo [7, 8].

A transição entre esses modelos lineares e não lineares exige uma forma robusta de avaliação, o que nos leva ao estudo das curvas ROC e da métrica AUC, detalhadas a seguir.

2.4 ROC e AUC

Na análise de desempenho de modelos de classificação, é fundamental considerar métricas além da acurácia. Essa medida pode apresentar resultados ilusórios em conjuntos de dados desbalanceados: em um cenário onde uma classe A representa 95% das amostras e uma classe B apenas 5%, um modelo que classifica todas as entradas como classe A atingiria 95% de acurácia sem, de fato, aprender qualquer padrão relevante [5]. Em aplicações reais, como o diagnóstico de doenças raras, ignorar a classe minoritária pode ter consequências críticas, tornando a acurácia uma métrica insuficiente para avaliar o desempenho do modelo.

Para contornar essa limitação, utiliza-se a curva ROC (*Receiver Operating Characteristic*), uma métrica originalmente binária que avalia a capacidade discriminativa do modelo por meio de duas medidas complementares [5]: o TPR (*true positive rate*), que indica a proporção de amostras positivas corretamente classificadas pelo modelo, e o FPR (*false positive rate*), que mensura a proporção de amostras negativas incorretamente classificadas como positivas. Ambas as medidas são calculadas variando-se o limiar de decisão do modelo — quanto maior esse limiar, maior a exigência de certeza para que uma amostra seja classificada como pertencente a uma determinada classe. Cada valor de limiar gera um par (FPR, TPR) que corresponde a um ponto no gráfico, cujo eixo y representa o TPR e o eixo x representa o FPR. A conexão desses pontos forma a curva ROC, permitindo analisar visualmente o comportamento do modelo em diferentes configurações de limiar [2].

A partir dessa curva, define-se o AUC (*Area Under the Curve*), que representa numericamente a área sob a curva ROC [5]. Quanto maior o AUC, melhor é a capacidade do modelo de discriminar as classes. Um valor de AUC igual a 1.0 indica um classificador perfeito, enquanto um valor de 0.5 equivale a um modelo sem capacidade discriminativa, cujas predições são equivalentes a uma escolha aleatória. Valores abaixo de 0.5 indicam um desempenho inferior ao acaso, sinalizando que o modelo está sistematicamente invertendo as predições [1].

Por ser intrinsecamente uma métrica binária, o ROC/AUC requer adaptações para ser aplicado em problemas multiclasse [6]. A estratégia mais comum é o OvR (*One-vs-Rest*), na qual cada classe é avaliada individualmente em contraste com todas as demais, tratadas coletivamente como uma única classe negativa. Essa abordagem gera um valor de AUC por classe, e a média desses valores fornece uma estimativa global do desempenho do modelo. Essa estratégia é especialmente relevante em contextos onde determinadas classes possuem maior importância clínica ou prática, como na distinção entre uma

doença grave e condições menos críticas, permitindo uma avaliação mais granular e informativa do desempenho dos classificadores estudados [6].

3 Resultados

Para evidenciar a diferença entre um classificador linear e um não linear, foi utilizado um conjunto de dados multiclasse sintético com três classes não linearmente separáveis, gerado com a função `make_gaussian_quantiles` do *scikit-learn*. A Figura 1 apresenta a comparação entre as curvas ROC/AUC dos modelos avaliados.

Os resultados mostraram que o KNN obteve desempenho superior ao Perceptron, com AUC micro de 0.998 e AUC macro de 0.998, enquanto o Perceptron apresentou AUC micro de 0.501 e AUC macro de 0.484. Esse comportamento está de acordo com a discussão teórica desenvolvida ao longo do relatório: como o Perceptron aprende apenas fronteiras lineares, sua capacidade de separação é limitada em distribuições curvas, ao passo que o KNN consegue se adaptar à vizinhança local das amostras.

Outras formas de avaliação, como matriz de confusão e fronteira de decisão, foram analisadas no notebook do experimento, disponível em: [Google Colab](#).

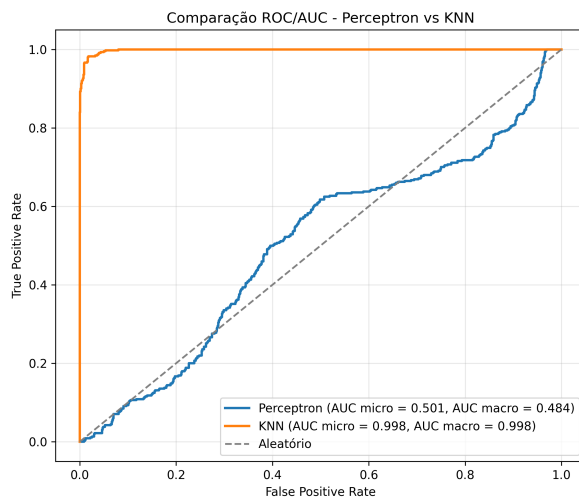


Figura 1: Comparação das curvas ROC/AUC entre Perceptron e KNN.

4 Conclusão

A elaboração deste estudo e a realização dos testes práticos deixaram claro que não existe um "melhor algoritmo" universal, mas sim o algoritmo mais adequado para cada tipo de dado. O Perceptron se mostrou um modelo muito interessante por sua simplicidade e facilidade de entender como os pesos são ajustados. No entanto, o fato de ele só conseguir traçar retas (ou hiperplanos) é uma limitação forte quando as classes estão muito misturadas ou formam padrões circulares.

Por outro lado, o KNN resolveu bem esse problema. Como ele não tenta achar uma equação matemática global, mas sim olhar para os "vizinhos" de cada ponto, ele conseguiu se moldar perfeitamente à distribuição mais complexa dos dados, o que foi comprovado pelo seu excelente resultado na métrica AUC.

Por fim, o estudo também mostrou a importância de saber avaliar os modelos corretamente. Se tivéssemos olhado apenas para a acurácia em um cenário desbalanceado, poderíamos ter sido enganados sobre o real desempenho dos algoritmos. O uso da curva ROC e do AUC foi fundamental para ter certeza de que o KNN realmente aprendeu a separar as classes, conectando a teoria vista no relatório com o comportamento prático dos modelos.

Tabela 1: Comparação dos principais hiperparâmetros do Perceptron e do KNN.

Modelo	Hiperparâmetro	Descrição e Impacto
Perceptron	<code>eta0</code>	Taxa de aprendizado constante. Controla a magnitude da atualização dos pesos a cada erro. Valores altos aceleram o treino mas podem causar instabilidade; valores baixos tornam a convergência lenta mas mais estável.
Perceptron	<code>max_iter</code>	Número máximo de épocas (passadas completas pelo dataset) que o algoritmo executará antes de parar, mesmo sem convergir.
Perceptron	<code>tol</code>	Tolerância para parada. Se a melhoria na função de perda for menor que este valor entre épocas, o treinamento é interrompido para evitar processamento desnecessário.
Perceptron	<code>early_stopping</code>	Estratégia de parada baseada em validação. Divide os dados internamente para validar o modelo a cada época; se o desempenho na validação não melhorar, o treino para para evitar overfitting.
Perceptron	<code>shuffle</code>	Se <i>True</i> , embaralha as amostras a cada época. Essencial para evitar que o modelo aprenda a ordem dos dados e melhore a generalização.
Perceptron	<code>class_weight</code>	Atribui pesos diferentes às classes. Útil em datasets desbalanceados, penalizando mais o erro na classe minoritária (ex: diagnóstico de doenças raras).
Perceptron	<code>fit_intercept</code>	Se <i>True</i> , adiciona um termo de viés (intercepto) à função de decisão, permitindo que a fronteira não passe necessariamente pela origem.
Perceptron	<code>penalty</code>	Tipo de regularização (L1, L2 ou Elastic Net) aplicada aos pesos para controlar a complexidade do modelo e prevenir overfitting.
KNN	<code>n_neighbors</code> (k)	Define a quantidade de vizinhos que influenciarão a classificação. Um k muito pequeno (ex: 1) torna o modelo sensível a ruídos (overfitting), enquanto um k muito grande suaviza demais a fronteira, podendo ignorar padrões locais (underfitting).
KNN	<code>weights</code>	Define a influência de cada vizinho. No modo <i>uniform</i> , todos os vizinhos têm o mesmo peso. No modo <i>distance</i> , vizinhos mais próximos têm um peso maior na votação do que os mais afastados.
KNN	<code>metric</code>	Define o cálculo de distância. A Euclidiana (linha reta) é a mais comum. A Manhattan (soma das diferenças absolutas) é útil em espaços em grade. A Minkowski é uma generalização que permite alternar entre as duas via parâmetro p .
KNN	<code>leaf_size</code>	Afeta a estrutura de árvore (KD-Tree ou Ball Tree) usada para armazenar os dados. Impacta apenas a velocidade de consulta e o uso de memória, não interferindo na precisão dos resultados.

Referências

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.
- [3] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [4] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [5] T. Fawcett, “An introduction to roc analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [6] D. J. Hand and R. J. Till, “A simple generalisation of the area under the roc curve for multiple class classification problems,” *Machine Learning*, vol. 45, no. 2, pp. 171–186, 2001.
- [7] Scikit-learn Developers. (2026) Perceptron. Acessado em: 30 de abril de 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html
- [8] ——. (2026) Knn. Acessado em: 30 de abril de 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html#>